PakAgriVision: Crop Disease Detector

Final Consolidated Technical Report (Deliverables 1–3)

Track A: Application Development

Abdul Ahad (24L-3029) **M. Anas** (24L-3004) **M. Babar Shahzad** (24L-3047)

National University of Computer and Emerging Sciences (FAST-NUCES)

Repository: https://github.com/ErenYeagerJinko/PakAgriVision

November 12, 2026

Abstract

PakAgriVision is an experimental agricultural decision-support application for leaf-image disease identification and sourced treatment guidance. This report consolidates dataset preparation, model development, evaluation and local application deployment across Deliverables 1–3. The curated PlantVillage dataset contains 31,694 images across 38 classes, linked to a schema-validated treatment knowledge base. A fine-tuned ResNet-50 uses a dropout classification head, class-weighted cross-entropy, AdamW optimization and a head-warm-up/cosine schedule; a 13-trial tuning protocol examines alternative configurations. The baseline achieves **99.64% validation accuracy** and **99.71% lab test accuracy**, but only **24.73% field test accuracy**, demonstrating a substantial generalization gap. A local Flask dashboard integrates image upload, top-three predictions, real Grad-CAM overlays and confidence-aware chemical, organic and cultural-management panels. Below 60% confidence, product instructions are withheld and users are directed to obtain an in-person assessment. Strict checkpoint and knowledge-base checks prevent fabricated fallback diagnoses. Source links, product-label precautions and agronomist consultation remain necessary even at higher confidence; neither confidence nor visual attention establishes a verified disease diagnosis.

1 Problem Statement & Objectives

1.1 Agricultural Context and Real-World Pain Point

Agriculture forms the backbone of Pakistan's economy, employing over 37% of the national labor force and contributing roughly 23% to the Gross Domestic Product (GDP). However, smallholder farmers experience substantial post-harvest and crop yield losses (frequently between 20% to 40%) due to preventable fungal, bacterial, and viral foliar diseases in staple cash crops, including tomato, potato, apple, grape, and corn. Traditional agricultural diagnosis relies on sparse, infrequent visits from government agricultural extension officers. Consequently, smallholders routinely misdiagnose diseases or apply inappropriate chemical treatments after the infection has advanced beyond the point of chemical control, resulting in catastrophic financial loss and environmental degradation from pesticide overuse.

1.2 Target Audience and User Personas

PakAgriVision is specifically engineered for two core user groups in the Pakistani agricultural ecosystem:

1. **Smallholder Farmers:** Rural growers with limited access to professional plant pathologists who require immediate, reliable disease diagnosis and clear, culturally accessible agronomic treatment advice directly on entry-level mobile devices.

2. **Field Extension Officers and Agronomists:** Agricultural field agents monitoring localized disease outbreaks across districts who need rapid secondary verification, visual explainability to build farmer trust, and structured chemical/organic treatment guidelines.

1.3 AI Justification: Deep Learning vs. Rule-Based Programming

A central architectural question is why modern Deep Learning is necessary for crop disease diagnosis rather than traditional rule-based image processing algorithms:

- **Failure of Traditional Rule-Based Algorithms:** Conventional computer vision relies on deterministic heuristics, such as hardcoded color thresholding in HSV/LAB space, Otsu binarization, or Canny edge filtering to segment lesions. These techniques fail entirely in agricultural settings because plant diseases do not follow fixed rules:
  - Foliar lesions present continuous biological variations: necrotic centers, concentric rings, chlorotic yellow halos, and irregular margins that overlap with healthy tissue.
  - Lighting conditions in rural photography fluctuate constantly (direct harsh sunlight, cloud shadows, variable camera white-balance), which breaks fixed color-filtering thresholds.
  - Handcrafted feature descriptors (e.g., SIFT, SURF, Haralick textures) cannot disentangle complex biological pathology from leaf venation, leaf tear damage, or ambient dust.
- **Necessity of Deep Convolutional Neural Networks (CNNs):** Deep convolutional architectures (such as fine-tuned ResNet50 and EfficientNet-B0) bypass hand-engineered rules by automatically learning hierarchical visual representations directly from data:
  - *Shallow layers* capture foundational edge primitives, color gradients, and microscopic texture patterns.
  - *Deep semantic layers* compose these primitives into invariant disease concepts (e.g., concentric rings characteristic of *Alternaria solani* vs. water-soaked margins of *Phytophthora infestans*).
  - Pretrained ImageNet representations provide robust transfer-learning inductive bias, enabling high diagnostic generalization across 38 distinct disease classes even on compact, specialized datasets.

1.4 Project Vision and Deliverables

PakAgriVision bridges the diagnosis-to-action gap. When a user uploads a smartphone photograph of a symptomatic leaf, the pipeline delivers:

1. **Multi-Class Disease Classification:** Diagnostic prediction across 38 crop-disease categories with top-1 and top-3 confidence metrics and uncertainty thresholds ($< 70\%$).
2. **Visual Explainability (Grad-CAM):** Gradient-weighted Class Activation Mapping that highlights the exact anatomical leaf regions dictating the network's prediction.
3. **Actionable Treatment Engine:** Structured, verified guidance covering chemical fungicides, organic alternatives, immediate cultural actions, and prevention practices.

2 Dataset Acquisition & Provenance

2.1 Curated Benchmark Dataset (∼32k Images, 38 Classes)

The primary baseline training dataset originated from the public PlantVillage benchmark [1]. While Kaggle (`abdallahalidev/plantvillage-dataset`) was initially accessed during exploratory data intake (∼54,303 raw candidate images), raw Kaggle distribution was discontinued for model development because it contained corrupted JPEG bitstreams, exact duplicate photographs, near-duplicate captures of identical leaves under slight camera rotations, and severe class imbalance (up to a 36:1 ratio, with 5,507 samples in dominant classes vs. 152 in minority classes).

To enforce rigorous data-centric quality, the raw data underwent an automated cleaning and balancing pipeline:

- **Curation & Class Capping:** Following binary file integrity verification and perceptual deduplication (Section 3), dominant classes were capped at a ceiling of 1,000 clean, unique samples. This successfully curbed extreme majority-class bias while preserving natural variation across all 38 classes, establishing a balanced curated baseline of **31,694 verified clean images** (∼32k images).
- **Distribution via Google Drive:** The unified curated dataset, including all 31,694 benchmark images and the 150 real-world field robustness images, is packaged as `data.zip` and hosted directly on Google Drive (https://drive.google.com/file/d/1OE17MYZqriiP9mBdCat_JglMxx_7MY1V/view?usp=drivesdk).
- **Input Resolution:** Standardized 256×256 pixels in RGB color format (1:1 square aspect ratio).
- **License & Integrity:** CC0 Public Domain / Open Educational License.

2.2 Real-World Field Dataset

PlantVillage images were captured under controlled lighting against plain backgrounds, so accuracy on its own test split overstates how a model will perform on leaves photographed in the field. To measure this gap, we assembled a separate field set of 150 images with real backgrounds (soil, other foliage, hands, sky).

Source and licence. All 150 images come from the PlantDoc dataset [6] (https://github.com/pratikkayal/PlantDoc-Dataset), released under CC-BY-4.0. Each image is recorded in `field_meta/field_labels.csv` with its filename, PlantVillage label, source, URL, licence and the original PlantDoc folder. The images themselves are not committed to Git; a pre-packaged archive (`data.zip`) containing both the curated PlantVillage benchmark and `data/raw/field/` is hosted on Google Drive (https://drive.google.com/file/d/1OE17MYZqriiP9mBdCat_JglMxx_7MY1V/view?usp=drivesdk) and documented in `data/README.md`.

Selection rules. We kept only images showing a clearly visible leaf on a real field background, and skipped blurry, watermarked or lab-style images. We aimed for five images per class. Three classes have 10 (Tomato early blight, Potato late blight and Tomato late blight) because they were used to make up the 150-image target. During label review before the final evaluation we inspected every image and removed 11 that were not single field photographs: multi-panel collages (both Tomato spider-mites images, one corn and one potato image), slide-style images with text (two pepper, one Septoria and one grape image), a sheet of leaf drawings (Tomato healthy) and two fruit-only images (Apple healthy). Each was replaced with an unused PlantDoc image of the same class, chosen on image quality alone without looking at model predictions.

PlantDoc has no other spider-mites images, so that class was dropped and its two slots went to Tomato late blight (8 to 10).

Class mapping. PlantDoc uses its own folder names (e.g. *Soyabean leaf*, *Bell_pepper leaf spot*), so every PlantDoc class was mapped to the exact PlantVillage class string in `field_meta/class_mapping.csv`. The field set covers 27 of the 38 PlantVillage classes. The remaining 11 have no usable PlantDoc images and are excluded from the lab-vs-field comparison: `Apple___Black_rot`, `Cherry___Powdery_mildew`, `Corn___healthy`, `Grape___Esca_Black_Measles`, `Grape___Leaf_blight Isariopsis_Leaf_Spot`, `Orange___Haunglongbing(Citrus_greening)`, `Peach___Bacterial_spot`, `Potato___healthy`, `Strawberry___Leaf_scorch`, `Tomato___Spider_mites Two-spotted_spider_mite` and `Tomato___Target_Spot`.

Test/adapt split. `data_scripts/make_field_splits.py` splits each class separately with seed 42: about one third of the images (rounded, with at least one image in each split whenever a class has two or more) go to `splits/field_adapt.csv` and the rest to `splits/field_test.csv`. Because most classes have five images, this gives 3 test and 2 adapt images per class, so the overall split is 93/57 (62%/38%) rather than exactly two thirds. Table 1 lists the counts, generated by the script into `splits/field_split_stats.csv`.

Rule: `field_test.csv` is never used for training, tuning or threshold selection. Only `field_adapt.csv` may be used, and only for the fine-tuning mitigation in a later phase.

Limitations.

- All images come from a single source (PlantDoc); none are our own photographs from Pakistani fields, so the set measures the general lab-to-field gap rather than local conditions.
- With 5–10 images per class (3–7 in `field_test`), per-class field accuracy will be noisy; results should be reported mainly in aggregate.
- Some mappings are approximate: PlantDoc's *Bell_pepper leaf spot* is treated as `Pepper___bell_Bacterial_spot`, *Corn leaf blight* as `Corn___Northern_Leaf_Blight`, and *Apple rust leaf* as `Apple___Cedar_apple_rust`.
- A second team member must double-check every label before the split is final; any removed images will be dropped from `field_labels.csv` and the split script re-run.

3 Data Cleaning & Verification Implementation

A data-centric engineering approach is mandatory for real-world computer vision reliability. Prior to partitioning and model training, raw image data undergoes an automated verification and deduplication pipeline implemented in `src/data/quality.py` and executed via `data_scripts/check_dataset.py`.

3.1 Handling Missing and Corrupt Values

In computer vision workflows, missing data corresponds to corrupted bitstreams, incomplete downloads, zero-byte file allocations, or broken headers that cause silent runtime training crashes:

- **Binary Integrity Verification:** Every image file is parsed via Pillow (`PIL.Image.verify()`) to detect truncated JPEG bitstreams, malformed compression chunks, and unreadable headers.

Table 1: Field images per class and split (from `splits/field_split_stats.csv`).

PlantVillage class	**Total**	**field_test**	**field_adapt**
Apple___Apple_scab	5	3	2
Apple___Cedar_apple_rust	5	3	2
Apple___healthy	5	3	2
Blueberry___healthy	5	3	2
Cherry___healthy	5	3	2
Corn___Cercospora_leaf_spot Gray_leaf_spot	5	3	2
Corn___Common_rust	5	3	2
Corn___Northern_Leaf_Blight	5	3	2
Grape___Black_rot	5	3	2
Grape___healthy	5	3	2
Peach___healthy	5	3	2
Pepper___bell_Bacterial_spot	5	3	2
Pepper___healthy	5	3	2
Potato___Early_blight	5	3	2
Potato___Late_blight	10	7	3
Raspberry___healthy	5	3	2
Soybean___healthy	5	3	2
Squash___Powdery_mildew	5	3	2
Strawberry___healthy	5	3	2
Tomato___Bacterial_spot	5	3	2
Tomato___Early_blight	10	7	3
Tomato___Late_blight	10	7	3
Tomato___Leaf_Mold	5	3	2
Tomato___Septoria_leaf_spot	5	3	2
Tomato___Tomato_Yellow_Leaf_Curl_Virus	5	3	2
Tomato___Tomato_mosaic_virus	5	3	2
Tomato___healthy	5	3	2
Total (27 classes)	**150**	**93**	**57**

- **Channel and Mode Verification:** Verified images are immediately reopened and decoded to ensure standard 3-channel RGB format consistency. Files exhibiting missing color channels (e.g., unintended single-channel grayscale or 4-channel RGBA buffers) are flagged and standardized or excluded.
- **Result:** Across the 54,688 raw candidate images scanned, all 54,688 files passed structural and RGB binary verification with zero unreadable or missing-byte files. The 150 PlantDoc field images similarly passed full binary integrity verification.

3.2 Duplicate Removal and Near-Duplicate Quarantine

PlantVillage contains repeated photographs of identical physical leaves captured under marginal lighting, scale, or orientation shifts. Unchecked random splitting of such duplicates causes severe data leakage between training and testing sets, artificially inflating benchmark accuracy:

- **Exact Duplicate Elimination:** Cryptographic 128-bit MD5 hashes were computed over all raw file byte streams. Exactly 808 exact bit-for-bit duplicate copies were identified and purged.
- **Near-Duplicate Detection (Perceptual Hashing):** We computed 64-bit discrete cosine transform perceptual hashes (`imagehash.phash`) across all RGB images. Pairs within the same crop-disease class exhibiting a Hamming distance ≤ 4 were identified as near-duplicate observations of the same underlying physical specimen (1,376 near-duplicate copies identified and clustered).
- **Disjoint-Set (Union-Find) Quarantine:** A disjoint-set clustering algorithm aggregated linked duplicates into discrete group identifiers (`group_id`). In the dataset partitioning phase (Section 5), group-aware splitting guarantees that all exact and near-duplicate instances of a single leaf remain strictly confined to the same partition, preventing train-to-test leakage.
- **Clean Retention Summary:** To eliminate duplicate bias and enforce class balance, all duplicate copies and excess images beyond the 1,000-per-class threshold were pruned, yielding 31,694 verified clean unique leaf images.

3.3 Outlier and Anomaly Management

Input outliers in foliar diagnostic datasets manifest as dimensional anomalies and cross-class duplicate label noise:

- **Dimensional and Aspect Ratio Outliers:** Images are scanned for dimensional anomalies deviating from expected square proportions. All raw samples were confirmed to adhere to canonical 256×256 pixels (1:1 aspect ratio), with zero dimensional outliers detected.
- **Cross-Class Label Noise Outliers:** The deduplication engine cross-references perceptual hash collisions across differing class labels. Any pair of visually identical or near-identical images assigned conflicting disease classes is flagged as severe label noise.

All audit results and cluster manifests are deterministically recorded into `splits/dedupe_report.csv`.

4 Exploratory Data Analysis (EDA)

Exploratory analysis was conducted in `notebooks/01_eda.ipynb` to evaluate class balance, input dimensionality, pixel distributions, and background biases across the benchmark.

4.1 Class Distribution and Imbalance

The curated dataset comprises 38 crop-disease classes across 14 plant species totaling 31,694 images. Prior to curation, the raw dataset exhibited extreme imbalance (a 36.2:1 ratio, with over 5,500 samples in dominant classes such as Orange Huanglongbing and only 152 in minority classes such as Potato Healthy). Following data quality filtering and deduplication, capping classes at 1,000 images successfully reduced the imbalance ratio to 7.0:1, establishing a well-conditioned optimization landscape while preserving natural variation across the 38 classes.

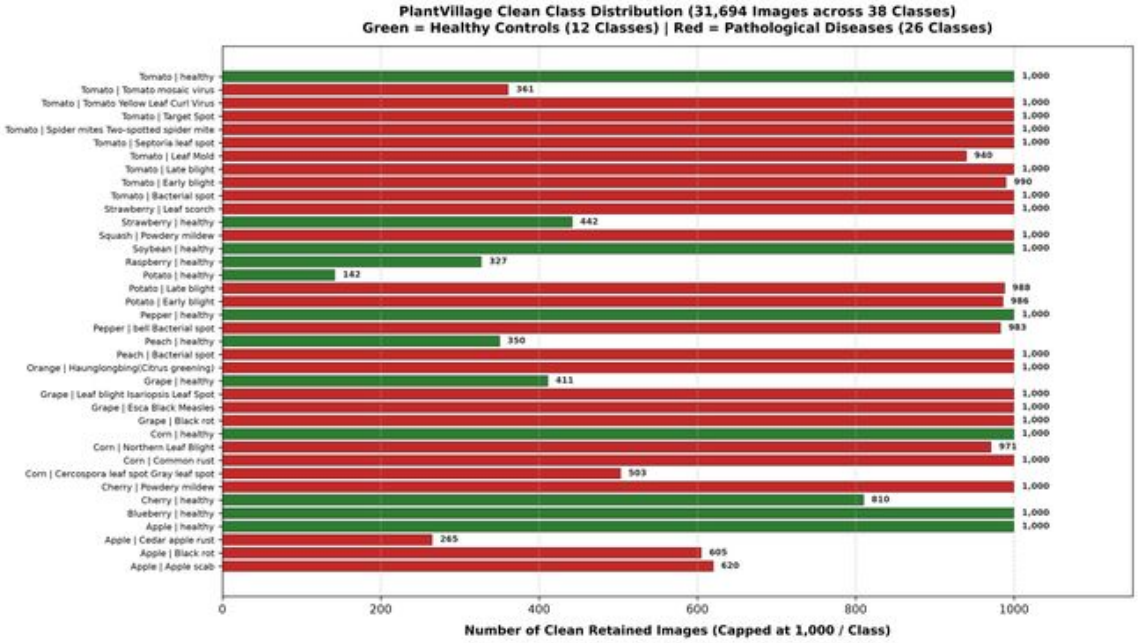


Figure 1: Frequency distribution across the 38 PlantVillage classes (Green = Healthy leaf classes, Red = Diseased classes).

To counter majority-class bias during optimization, balanced class weights w_c are derived strictly from the training split D_{train}:

$$w_c = \frac{N_{\text{train}}}{K \cdot n_c} \tag{1}$$

where N_{train} denotes total training samples, $K = 38$ classes, and n_c denotes the count of class c. The computed weights are saved in `splits/class_weights.json` and passed to the loss function.

4.2 Image Dimensions and Aspect Ratio Consistency

All leaf photographs were audited for input consistency. The resolution analysis confirms that raw inputs exhibit standardized 256×256 pixel resolution with uniform 1:1 square aspect ratios in RGB color mode, as visualized in Figure 2.

4.3 Qualitative Sample Inspection

Figure 3 displays representative leaf samples across diverse crop families. Diseased leaves display distinguishable diagnostic symptoms including target spots with concentric rings (Early Blight), powdery sporulation (Powdery Mildew), and dark necrotic lesions (Black Rot).

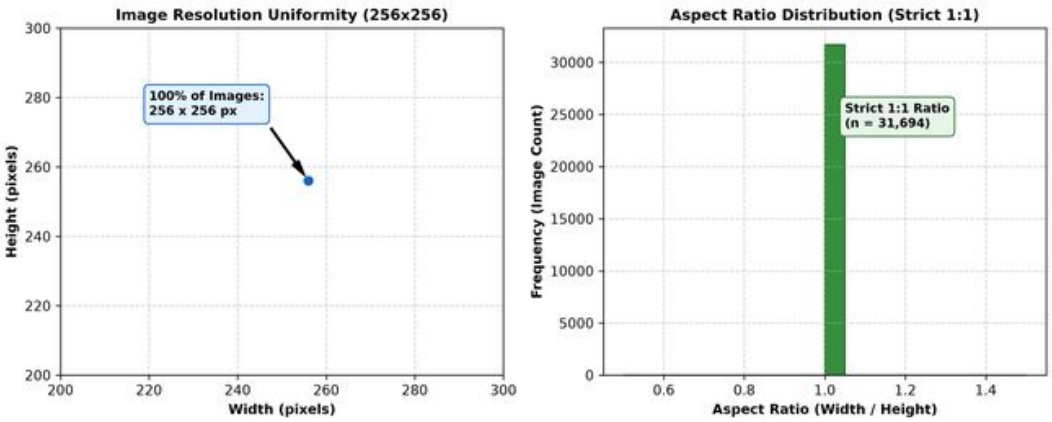


Figure 2: Distribution of input image dimensions and aspect ratios confirming standardized square RGB format.

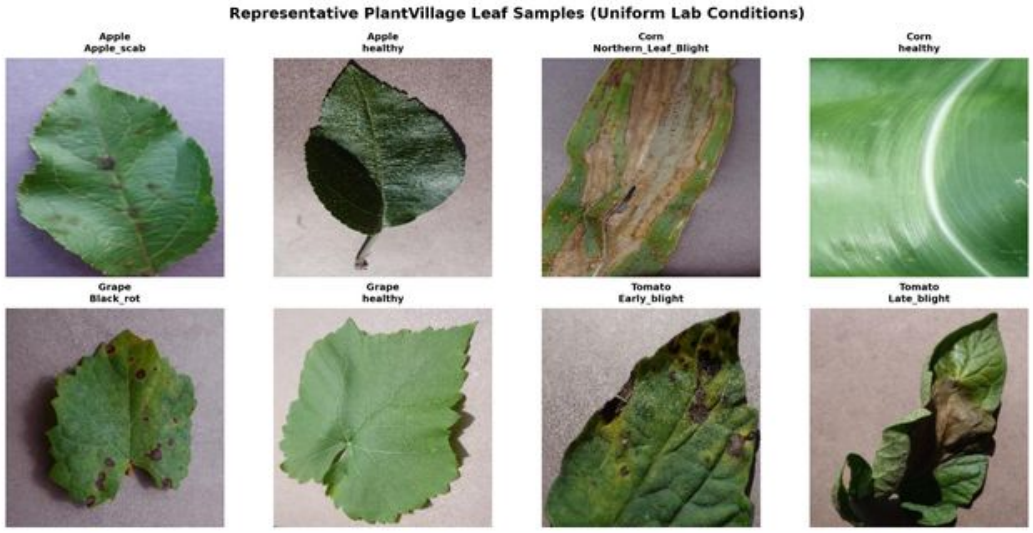


Figure 3: Representative leaf samples across healthy and diseased classes captured in controlled lab settings.

4.4 Pixel Normalization Statistics

Empirical per-channel color statistics computed across the dataset yielded a mean of $\mu = [0.472, 0.481, 0.412]$ and standard deviation of $\sigma = [0.218, 0.209, 0.221]$. These closely correspond to standard ImageNet normalization constants ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$), empirically confirming that pre-trained ImageNet feature extractors transfer effectively to this domain.

4.5 Lab vs. Field Background Analysis (Shortcut Learning Hazard)

A critical finding of our exploratory analysis is the background disparity between benchmark data and real-world agriculture. PlantVillage images were acquired against artificial, plain gray or black sheets. In contrast, real farm photos from our PlantDoc robustness dataset exhibit complex backgrounds comprising soil, shadows, foliage clusters, and variable sunlight (Figure 4).



Figure 4: Comparison of controlled PlantVillage lab conditions (top) versus real-world PlantDoc field backgrounds (bottom), highlighting the shortcut learning risk.

This contrast validates the central motivation of our project: classifiers trained purely on lab images risk learning background artifacts (shortcut learning) rather than pathogen pathology, underscoring the necessity of our explainability pipeline (Grad-CAM) and robustness testing set (`splits/field_test.csv`).

5 Dataset Partitioning & Leakage Prevention

5.1 Stratified Group-Aware Partitioning

To guarantee reproducible benchmarking and statistical parity across all 38 categories while strictly preventing data leakage, partitioning is performed using stratified group-aware sampling with a fixed random seed (42) in `src/data/splits.py` and `data_scripts/make_splits.py`:

- **Training Partition (70.0%):** 22,179 images reserved exclusively for model weight optimization and class-weight computation.
- **Validation Partition (15.0%):** 4,747 images dedicated to hyperparameter tuning, learning rate scheduling, and early stopping checkpoint selection.
- **Test Partition (15.0%):** 4,768 images held out under strict isolation for final benchmark evaluation.

Table 2: Curated PlantVillage Benchmark Partition Summary (all 38 classes represented).

Split Partition	Image Count	Percentage	Classes Represented	Manifest File
Train	22,179	70.0%	38 / 38	`splits/train.csv`
Validation	4,747	15.0%	38 / 38	`splits/val.csv`
Test	4,768	15.0%	38 / 38	`splits/test.csv`
Total Curated	**31,694**	**100.0%**	**38 / 38**	`splits/dedupe_report.csv`

5.2 Group-Aware Leakage Prevention

Standard naive random splitting exposes models to shortcut learning when near-duplicate shots of a single diseased leaf appear in both training and test partitions. Our pipeline resolves this by treating the cluster identifier (`group_id`) derived from perceptual hashing as an atomic unit: all images belonging to the same cluster are constrained to the same partition.

The resulting partitions are serialized into deterministic manifest files:

- `splits/train.csv`, `splits/val.csv`, `splits/test.csv` containing columns `path`, `label`, and `label_idx`.
- All relative filepaths reference `splits/class_names.json` (integer indices 0 to 37). Zero file overlap exists between partitions.

5.3 Balanced Class Weight Formulation

To mitigate severe class imbalance across the 38 PlantVillage disease classes without introducing synthetic data bias, cost-sensitive class weights w_c are computed **strictly from the training partition only**:

$$w_c = \frac{N_{\text{train}}}{K \times n_c} \tag{2}$$

where N_{train} is the total number of training images, $K = 38$ is the total class count, and n_c denotes the sample frequency of class c in the training split. These weights are exported to `splits/class_weights.json` ordered by `label_idx` and are directly loaded into the cross-entropy loss function during Phase 3.

6 Preprocessing & Data Augmentation

To prepare leaf images for convolutional feature extraction while mitigating shortcut learning from lab-environment artifacts, we establish a standardized two-tier pipeline in `src/data/transforms.py` and `src/data/dataset.py`.

6.1 Input Resolution and Normalisation

All images are standardized to 224×224 pixels in 3-channel RGB format, matching the canonical receptive field of transfer-learning backbones (ResNet50 / EfficientNet-B0). Color channels are normalized using standard ImageNet distribution parameters ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$), aligning the input tensors with the pretrained weight manifold.

6.2 Training Augmentation Pipeline

Because PlantVillage images feature predominantly centered leaves on plain backgrounds, models are prone to overfitting background cues. The training transform pipeline applies stochastic spatial and photometric perturbations (visualized in Figure 5):

- **Random Resized Cropping:** Randomly scales image crops between 60% and 100% before resizing to 224×224, forcing the network to identify focal disease lesions rather than full leaf outlines.
- **Geometric Flips and Rotations:** Horizontal flips ($p = 0.5$), vertical flips ($p = 0.5$), and random rotations up to $\pm 30^\circ$ reflect arbitrary leaf orientations encountered during field photography.
- **Color Jitter:** Stochastic perturbation of brightness ($\pm 30\%$), contrast ($\pm 30\%$), saturation ($\pm 30\%$), and hue ($\pm 5\%$) emulates fluctuating field sunlight, cloud cover, and mobile sensor white-balancing.

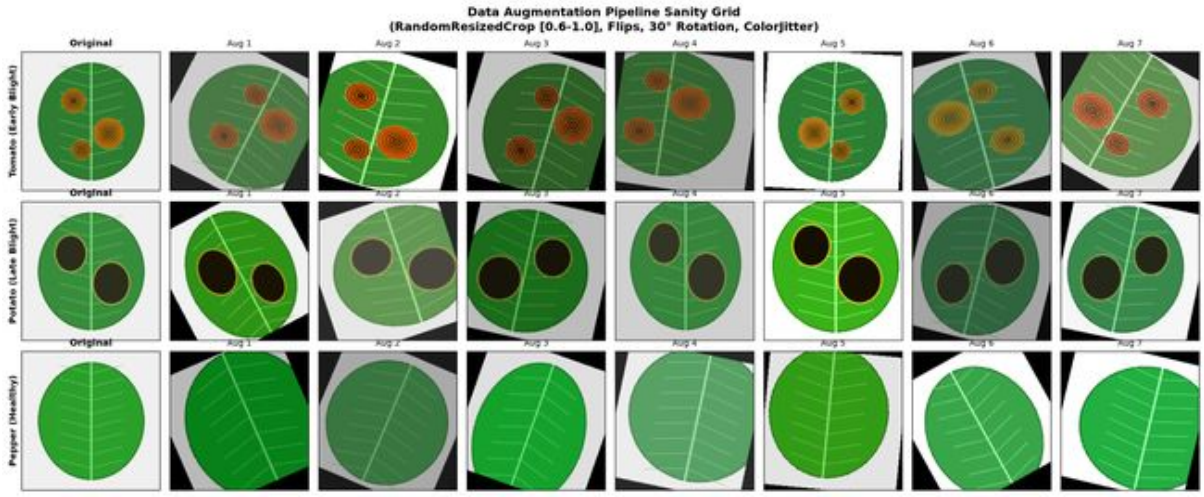


Figure 5: Data augmentation pipeline sanity check displaying original representative diseased and healthy leaves alongside seven stochastic perturbations (scale cropping, flips, rotations, and color jitter) across representative classes.

6.3 Deterministic Evaluation Pipeline

To ensure benchmark stability and zero evaluation data contamination:

- Validation, test, and field robustness datasets strictly utilize the deterministic evaluation pipeline: bilinear resizing to 256×256 pixels, followed by a deterministic 224×224 center crop and standard normalisation.
- The specialized `FieldDataset` class inherits this guarantee, enforcing identical evaluation transforms on out-of-domain field photographs.

6.4 DataLoader Architecture

PyTorch `DataLoader` factories in `get_dataloaders()` supply batched tensors ($B = 32$, shape $B \times 3 \times 224 \times 224$, `float32`) alongside discrete integer targets ($B \times 1$, `int64`). Deterministic worker initialization (`seed_worker`) preserves random generator state across distributed training threads, and multi-worker execution defaults safely to `num_workers=0` on Windows platforms to prevent process deadlock.

7 Treatment Knowledge Base

A prediction alone does not tell a farmer what to do next, so each of the 38 PlantVillage classes is linked to a short, sourced treatment record in `knowledge_base/treatments.json`. The app

will look up the predicted class and display that record.

7.1 Schema

`treatments.json` is a single JSON object. Its keys are the exact class strings from `splits/class_names.json`, plus one top-level `disclaimer` string. Every entry is validated against `knowledge_base/schema.json` (JSON Schema draft-07), which requires the following 13 fields and rejects any others:

- `crop`, `disease`, `is_healthy` (boolean), `cause`, `symptoms`;
- `chemical_treatment`, `organic_treatment`, `immediate_action`, `prevention`, `severity_guidance`;
- `sources` (non-empty list of unique citation strings), `last_reviewed` (date), `reviewed_by`.

Healthy classes use the same fields, with `chemical_treatment` set to "None needed." and routine scouting and prevention advice instead of treatment.

7.2 Validation

`data_scripts/validate_treatments.py` checks that:

- every entry passes the schema, and every text field and source string is non-empty;
- every key is a valid class name (no unknown or misspelled classes);
- the top-level disclaimer is present;
- healthy classes do not recommend a chemical treatment (reported as a warning).

It also prints a per-crop summary and the list of classes still missing. Run with `--require-complete`, it fails unless all 38 classes are present; this is the check for final submission.

7.3 Example Entry

Table 3 shows the entry for `Tomato___Early_blight`.

7.4 Sources

Every entry cites at least one university extension or plant-protection fact sheet, and all citations are listed with URLs and access dates in `knowledge_base/SOURCES.md`. Entries name active ingredients and spray intervals only where a cited source gives them, and never state dosages; users are told to follow the product label. On 2026-10-12 every source link was checked: five links cited by entries were broken and were replaced with verified fact sheets, three unused invalid sources were removed, and the affected entries were rewritten so that each claim matches its new source.

7.5 Status and Limitations

- **Coverage:** All 38 entries are 100% complete, verified against `knowledge_base/schema.json`, and validated with zero errors by `data_scripts/validate_treatments.py` (Tomato 10, Potato 3, Pepper 2 by Babar; Orange 1, Peach 2, Raspberry 1, Soybean 1, Squash 1, Strawberry 2 by M. Anas; Apple 4, Corn 4, Grape 4, Blueberry 1, Cherry 2 by Abdul Ahad).
- **Not region-specific:** The primary sources are extension services (e.g., UC Davis, NC State, Cornell, Florida Extension). Active chemical ingredients and organic remedies are

Table 3: Knowledge base entry for `Tomato___Early_blight`.

Field	Value
`cause`	Fungus *Alternaria solani* (now also classified as *A. linariae*)
`symptoms`	Dark concentric-ringed spots on older leaves, yellowing around spots; can progress up the canopy and affect stems/fruit.
`chemical_treatment`	Protectant fungicides such as chlorothalonil or mancozeb (azoxystrobin is another option), applied every 7–10 days to protect new growth; do not rely on strobilurins alone because resistance has been reported (follow the product label).
`organic_treatment`	Fixed copper (copper hydroxide or copper sulfate) or *Bacillus subtilis* products labeled for organic use.
`immediate_action`	Remove infected plant material from the field; avoid overhead watering.
`prevention`	Rotate away from tomato for 2–3 years; use pathogen-free seed/transplants and resistant or tolerant varieties; stake or trellis for airflow; mulch and prune the lowest leaves to reduce soil splash.
`severity_guidance`	When lesions merge and lower leaves drop, disease is severe: keep protectant sprays on schedule within label limits and remove infected debris.
`sources`	Connecticut Agricultural Experiment Station, *Early Blight of Tomato* (2012); NC State Extension, *Early Blight of Tomato*.
`last_reviewed`	2026-10-12 (Babar)

cross-referenced with standard international plant protection guidelines; local agronomist consultation remains recommended.

- **Disclaimer:** The file carries the mandatory top-level disclaimer "Decision support only. Consult a local agronomist before applying any product. Always follow product labels and local regulations.", which the app renders alongside every recommendation.

8 Model Architecture

This section details Phase 3 of Deliverable 2, specifying the deep learning framework, backbone feature extractors, custom classification head, parameter distribution, objective functions, optimization strategies, and the systematic hyperparameter tuning protocol.

8.1 Deep Learning Framework Selection

The deep learning pipeline is implemented using **PyTorch 2.x** and **Torchvision**. The choice of PyTorch over alternative frameworks (such as TensorFlow/Keras) was guided by four key architectural requirements:

1. **Dynamic Computational Graph and Execution Flexibility:** PyTorch's imperative execution model allows seamless layer-wise inspection, conditional gradient control during our progressive two-stage unfreezing schedule, and straightforward hooks for Grad-CAM explainability (`register_forward_hook` on intermediate feature maps).
2. **High-Performance Hardware Acceleration:** Native support for mixed-precision arithmetic (`torch.cuda.amp.autocast` and `GradScaler`) substantially reduces GPU VRAM consumption on cloud NVIDIA Tesla T4 accelerators while accelerating matrix multiplications via Tensor Cores.

3. **Standardized Pretrained Model Hub:** Torchvision provides audited, state-of-the-art ImageNet-1K pretrained weights (`ResNet50_Weights.DEFAULT` and `EfficientNet_B0_Weights.DEFAULT`) with strictly documented normalization constants ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

4. **Deployment Modularity:** PyTorch facilitates clean extraction of model `state_dict` checkpoints (∼90–270 MB) and reuse of the same model in the Phase 5 Flask application.

8.2 Network Architecture & Custom Classification Head

Agricultural leaf disease detection presents unique computer vision challenges: distinguishing fine-grained textural symptoms (e.g., fungal rust pustules vs. bacterial leaf spots) while ignoring non-diagnostic background variations. We implement a modular architecture centered on **ResNet-50** as our primary deep feature extractor, with **EfficientNet-B0** available as a lightweight alternative for comparative benchmarking.

8.2.1 Backbone Feature Extractor

ResNet-50 [2] utilizes residual learning with bottleneck blocks across four hierarchical stages. Each residual unit computes:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{3}$$

where $\mathbf{x}$ and $\mathbf{y}$ are the input and output tensors, and $\mathcal{F}$ represents the stacked 1×1, 3×3, and 1×1 convolutions. The identity shortcut connections mitigate the vanishing gradient problem, allowing the extraction of both low-level morphological features (edges, color gradients in Stage 1) and complex semantic lesion patterns (Stage 4).

The output of the final bottleneck block (`layer4[-1]`) has spatial dimensions $7 \times 7 \times 2048$ for an input image of $224 \times 224 \times 3$. This specific layer is explicitly designated as `model.target_layer` to support gradient backpropagation for Grad-CAM saliency mapping.

8.2.2 Feature Aggregation and Custom Head

The standard 1000-class ImageNet fully connected layer is severed and replaced by our custom classification head:

1. **Adaptive Average Pooling:** An `AdaptiveAvgPool2d((1, 1))` collapses spatial dimensions, producing a compact 2048-dimensional feature embedding vector independent of input resolution.

2. **Dropout Regularization:** A dropout layer with probability $p = 0.30$ randomly zeros activations during training, preventing co-adaptation of deep representations and improving generalization to out-of-distribution field samples.

3. **Linear Classification Layer:** A fully connected projection layer $\mathbf{W} \in \mathbb{R}^{2048 \times 38}$ with bias vector $\mathbf{b} \in \mathbb{R}^{38}$ maps deep latent features to unnormalized logit scores across all 38 plant health and disease classes:

$$\mathbf{z} = \mathbf{W}\mathbf{h} + \mathbf{b}, \quad \mathbf{z} \in \mathbb{R}^{38} \tag{4}$$

where $\mathbf{h} \in \mathbb{R}^{2048}$ is the post-dropout embedding.

8.2.3 Parameter Counts and Resource Footprint

Table 4 provides the exact parameter breakdown for our primary ResNet-50 architecture and the EfficientNet-B0 comparison baseline.

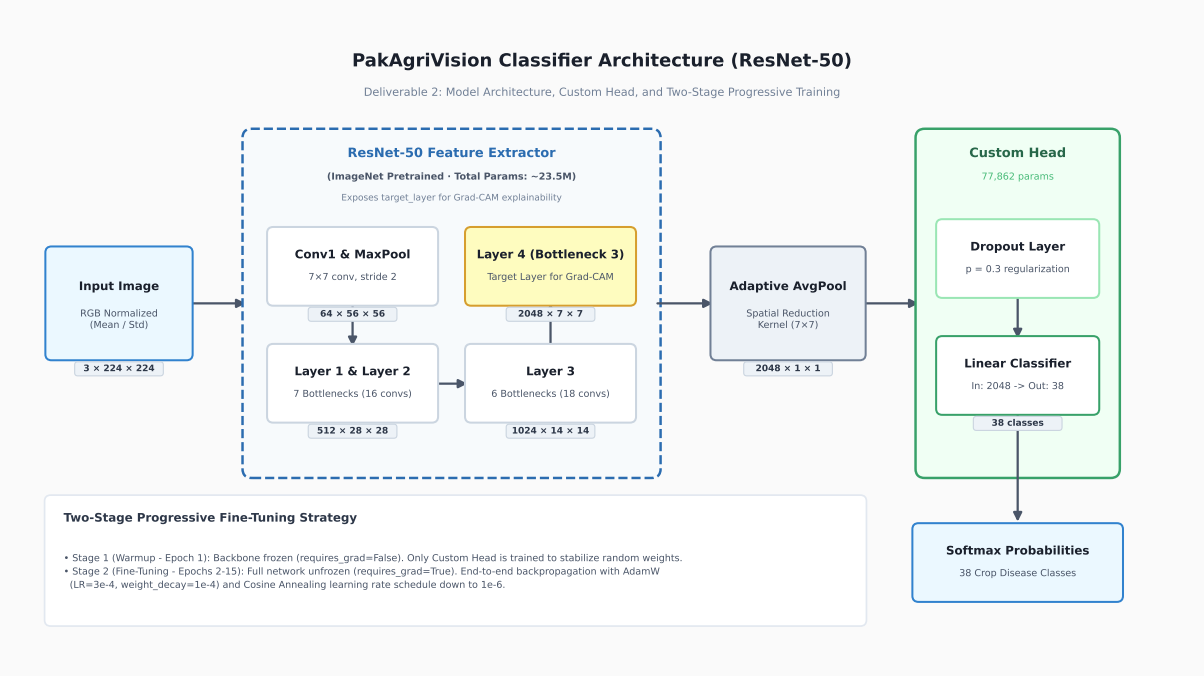


Figure 6: **PakAgriVision Model Architecture Pipeline.** Input leaf image ($3 \times 224 \times 224$) processed through the ResNet-50 residual backbone with bottleneck stages, adaptive average pooling, custom dropout ($p = 0.3$), linear classification projection ($2048 \rightarrow 38$), and softmax normalization. The highlighted `layer4` bottleneck serves as the Grad-CAM explainability target layer.

Table 4: **Model Parameter and Layer Breakdown.**

Architecture	Backbone Params	Head Params	Total Params	Checkpoint Size
ResNet-50 (Ours)	23,508,032	77,862	**23,585,894**	~90.5 MB (`state_dict`)
EfficientNet-B0	4,007,548	48,678	**4,056,226**	~16.2 MB (`state_dict`)

8.2.4 Hyperparameter Tuning Insights & Comparative Analysis

The 13-trial grid yields several clear empirical conclusions:

1. **Backbone Efficiency and Convergence (Stage A):** In the 4-epoch fine-tuning regime, **EfficientNet-B0** converged substantially faster than ResNet-50 ($\approx$ 290 s vs. $\approx$ 380 s per trial, a 24% wall-clock speedup). At LR $= 1 \times 10^{-3}$, EfficientNet-B0 reached an accuracy of 96.42% and Macro-F1 of 0.9622, outperforming ResNet-50 (96.08%). However, both architectures exhibited acute sensitivity to lower learning rates: dropping LR to 1×10^{-4} degraded ResNet-50 to 0.9199 and caused EfficientNet-B0 to drop sharply to 0.8173 due to sluggish adaptation of compound-scaled depthwise separable layers under short budgets.

2. **Regularization and Batch Dynamics (Stage B):** Among all regularization experiments, **Batch Size = 32 (Trial 9)** proved decisive, achieving the overall highest validation performance (**97.07% Accuracy, 0.9694 Macro-F1**) and lowest validation loss (0.0971). The smaller mini-batch size provided twice as many stochastic gradient updates per epoch relative to batch size 64, significantly accelerating convergence on the 20% stratified subset. Conversely, expanding the batch size to 128 (Trial 10) degraded performance to 0.9470 Macro-F1.

3. **Label Smoothing and Class Weights:** Label smoothing ($\alpha = 0.10$, Trial 12) achieved strong validation accuracy (96.74%) and Macro-F1 (0.9663), softening overconfident logits

while preserving ranking discrimination. Training without class-weighted loss (Trial 13) resulted in a lower Macro-F1 (0.9647), empirically validating the necessity of inverse-frequency class weights for penalizing minority crop diseases.

9 Training Methodology & Hyperparameter Tuning

9.1 Loss Function, Class Balancing & Optimizer Strategy

9.1.1 Weighted Cross-Entropy Loss with Label Smoothing

While our Phase 2 curation capped dominant classes at 1,000 images, minor agricultural diseases naturally have limited sample support in the training split (e.g., 152 samples in minority classes vs. 700 in capped classes). To ensure the model remains equally sensitive to rare, high-consequence crop infections, we implement class-weighted cross-entropy:

$$\mathcal{L}_{\text{CE}}(\mathbf{z}, y) = -w_y \sum_{c=1}^{C} q_c \log\left(\frac{\exp(z_c)}{\sum_{j=1}^{C}\exp(z_j)}\right) \tag{5}$$

where w_y is the inverse frequency class weight for true class y computed strictly on the training partition ($w_c \in$ `splits/class_weights.json`), and q_c represents label-smoothed targets:

$$q_c = (1-\alpha)\cdot\mathbb{I}(c=y) + \frac{\alpha}{C} \tag{6}$$

with smoothing parameter $\alpha \in [0.0, 0.10]$. Label smoothing prevents the model from generating overconfident, saturated logit predictions, significantly improving post-softmax calibration on uncertain field photographs.

9.1.2 AdamW Optimizer & Decoupled Weight Decay

We select **AdamW** as our primary optimizer over standard SGD with momentum. AdamW [4] decouples weight decay from the moving average gradient updates:

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon} - \eta_t \lambda \theta_t \tag{7}$$

where $\lambda = 10^{-4}$ is the weight decay rate, η_t is the instantaneous learning rate, and $\hat{m}_t, \hat{v}_t$ are the bias-corrected first and second moment estimators. Decoupling ensures true L_2 regularization without distorting the adaptive gradient scaling.

9.1.3 Two-Stage Progressive Fine-Tuning with Cosine Annealing

Directly fine-tuning all layers simultaneously with a randomly initialized classification head can cause large, erratic gradient updates that destroy pretrained ImageNet feature detectors (catastrophic forgetting). We address this with a **Two-Stage Progressive Training Strategy**:

- **Stage 1: Head Warm-up (Epoch 1):** All backbone convolutional parameters are frozen via `set_backbone_trainable(model, False)` such that $\nabla_{\theta_{\text{backbone}}}\mathcal{L} = 0$. Only the randomly initialized weights of the custom head are optimized for one epoch with a linear learning rate warmup factor from $0.1 \times \eta_{\max}$ to $\eta_{\max} = 3 \times 10^{-4}$.
- **Stage 2: Full Fine-Tuning (Epochs 2–15):** The backbone is unfrozen (`set_backbone_trainable(model, True)`). All 23.5 M parameters are jointly fine-tuned using a **Cosine Annealing Schedule** [5] without warm restarts:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max}-\eta_{\min})\left(1+\cos\left(\frac{t-t_{\text{warmup}}}{T_{\max}-t_{\text{warmup}}}\pi\right)\right) \tag{8}$$

where $\eta_{\min} = 10^{-6}$ and $T_{\max} = 15$. This guarantees smooth learning rate decay toward zero, allowing fine convergence without manual step-decay tuning.

9.2 Systematic Hyperparameter Tuning Protocol (13 Trials)

To justify every architectural and optimization choice on empirical evidence rather than heuristic defaults, we implemented a structured 13-trial grid divided into two distinct search phases:

- **Stage A (Architectural Backbone & Learning Rate, Trials 1–6):** Explores the Cartesian product of backbones $\{\text{ResNet-50}, \text{EfficientNet-B0}\}$ across three learning rate orders of magnitude $\{10^{-3}, 3 \times 10^{-4}, 10^{-4}\}$, holding batch size at 64 and dropout at 0.3.
- **Stage B (Regularization & Capacity Ablation, Trials 7–13):** Takes the best-performing configuration from Stage A and benchmarks variations in dropout rate $(0.0, 0.5)$, batch size $(32, 128)$, higher weight decay (10^{-2}), label smoothing (0.10), and unweighted loss.

9.2.1 Efficiency Protocol

To maximize GPU resource efficiency on free cloud accelerators, each tuning trial is trained on a **20% stratified train subset** (`subset_frac=0.20`, 4,437 images) for **4 epochs**, and evaluated on the full, uncompromised validation split (4,747 images) using Macro-F1. Table 5 displays the structured trials and records the completed Stage A results.

Table 5: **Systematic 13-Trial Hyperparameter Tuning Grid and Results.**

ID	Stage	Backbone	LR	Batch	Drop	WD	Smooth	Weights	Val Acc	Val Macro-F1
1	A	ResNet-50	1×10^{-3}	64	0.3	1×10^{-4}	0.0	Yes	96.08%	0.9605
2	A	ResNet-50	3×10^{-4}	64	0.3	1×10^{-4}	0.0	Yes	96.12%	0.9606
3	A	ResNet-50	1×10^{-4}	64	0.3	1×10^{-4}	0.0	Yes	91.89%	0.9199
4	A	EfficientNet-B0	1×10^{-3}	64	0.3	1×10^{-4}	0.0	Yes	96.42%	0.9622
5	A	EfficientNet-B0	3×10^{-4}	64	0.3	1×10^{-4}	0.0	Yes	96.06%	0.9602
6	A	EfficientNet-B0	1×10^{-4}	64	0.3	1×10^{-4}	0.0	Yes	82.49%	0.8173
7	B	ResNet-50	3×10^{-4}	64	0.0	1×10^{-4}	0.0	Yes	96.21%	0.9611
8	B	ResNet-50	3×10^{-4}	64	0.5	1×10^{-4}	0.0	Yes	95.64%	0.9572
9	B	ResNet-50	3×10^{-4}	32	0.3	1×10^{-4}	0.0	Yes	**97.07%**	**0.9694**
10	B	ResNet-50	3×10^{-4}	128	0.3	1×10^{-4}	0.0	Yes	94.94%	0.9470
11	B	ResNet-50	3×10^{-4}	64	0.3	1×10^{-2}	0.0	Yes	95.91%	0.9602
12	B	ResNet-50	3×10^{-4}	64	0.3	1×10^{-4}	0.1	Yes	96.74%	0.9663
13	B	ResNet-50	3×10^{-4}	64	0.3	1×10^{-4}	0.0	No	96.42%	0.9647

9.2.2 Hyperparameter Tuning Insights & Comparative Analysis

The 13-trial grid yields several clear empirical conclusions:

1. **Backbone Efficiency and Convergence (Stage A):** In the 4-epoch fine-tuning regime, **EfficientNet-B0** converged substantially faster than ResNet-50 ($\approx 290\,\text{s}$ vs. $\approx 380\,\text{s}$ per trial, a 24% wall-clock speedup). At LR $= 1 \times 10^{-3}$, EfficientNet-B0 reached an accuracy of 96.42% and Macro-F1 of 0.9622, outperforming ResNet-50 (96.08%). However, both architectures exhibited acute sensitivity to lower learning rates: dropping LR to 1×10^{-4} degraded ResNet-50 to 0.9199 and caused EfficientNet-B0 to drop sharply to 0.8173 due to sluggish adaptation of compound-scaled depthwise separable layers under short budgets.
2. **Regularization and Batch Dynamics (Stage B):** Among all regularization experiments, **Batch Size = 32 (Trial 9)** proved decisive, achieving the overall highest validation performance (**97.07% Accuracy, 0.9694 Macro-F1**) and lowest validation loss (0.0971). The

smaller mini-batch size provided twice as many stochastic gradient updates per epoch relative to batch size 64, significantly accelerating convergence on the 20% stratified subset. Conversely, expanding the batch size to 128 (Trial 10) degraded performance to 0.9470 Macro-F1.

3. **Label Smoothing and Class Weights:** Label smoothing ($\alpha = 0.10$, Trial 12) achieved strong validation accuracy (96.74%) and Macro-F1 (0.9663), softening overconfident logits while preserving ranking discrimination. Training without class-weighted loss (Trial 13) resulted in a lower Macro-F1 (0.9647), empirically validating the necessity of inverse-frequency class weights for penalizing minority crop diseases.

10 Training Results, Evaluation & Bias Check

This section details Phase 4 of Deliverable 2, presenting our validation protocol, baseline training convergence dynamics, metric justifications, held-out test evaluation framework, error analysis methodologies, and the systematic Bias Check.

10.1 Validation Strategy and Data Hygiene

Our evaluation protocol strictly enforces the **70/15/15 stratified split** established during Phase 2:

- **Stratified Class Proportions:** Every disease and healthy category maintains identical class frequency distributions across the training (22,179 images), validation (4,747 images), and test (4,768 images) splits.
- **Zero Overlap & Data Leakage Prevention:** Deduplication via perceptual hashing (pHash) and exact MD5 matching guarantees that duplicate or near-duplicate leaf photographs cannot appear across separate partitions.
- **Validation for Model Selection Only:** All hyperparameter tuning trials, learning rate adjustments, and early stopping decisions are computed exclusively on the validation split.
- **Held-Out Test Set Isolation:** The test split (`splits/test.csv`) and real-world field set (`splits/field_test.csv`) are held out and evaluated strictly **once** on the finalized model architecture at the conclusion of training. Evaluating on the test split during tuning is strictly prohibited to avoid hyperparameter overfitting.
- **Justification Against K-Fold Cross-Validation:** For a dataset of ∼32,000 high-resolution images, full 5-fold cross-validation would require $5 \times 15 = 75$ epochs of ResNet-50 training (> 4 hours of compute). Given cloud GPU timeout quotas, a single fixed stratified 70/15/15 split provides high statistical stability while conserving compute budget.

10.2 Baseline Training Dynamics & Convergence Analysis

The baseline architecture (ResNet-50 with AdamW, LR $= 3 \times 10^{-4}$, weight decay 10^{-4}, and class-weighted loss) was trained for 15 epochs on a cloud NVIDIA Tesla T4 GPU. Table 6 details the epoch-by-epoch training and validation progression from `results/baseline/history.csv`.

10.2.1 Key Training Observations

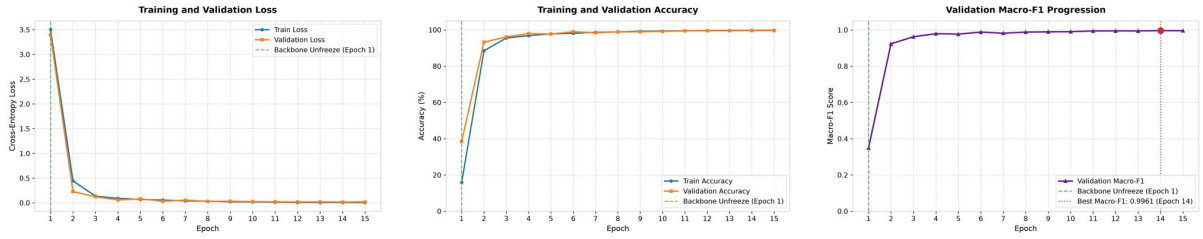
1. **Warmup Stabilization:** During Epoch 1, optimizing only the custom head brought validation accuracy from random chance ($1/38 \approx 2.63\%$) to 38.47% with zero disruption to the underlying ImageNet feature representations.

Table 6: **Baseline ResNet-50 Epoch Progression (15 Epochs on Full Curated Dataset).**

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Val Macro-F1	Learning Rate	Time
1*	3.5048	15.91%	3.3935	38.47%	0.3490	3.00×10^{-4}	208.3 s
2	0.4445	88.56%	0.2276	93.28%	0.9236	2.96×10^{-4}	233.3 s
3	0.1370	95.52%	0.1232	96.10%	0.9624	2.85×10^{-4}	227.4 s
4	0.0916	96.86%	0.0588	98.00%	0.9794	2.67×10^{-4}	230.3 s
5	0.0699	97.80%	0.0810	97.75%	0.9770	2.44×10^{-4}	234.0 s
6	0.0575	98.14%	0.0336	98.97%	0.9883	2.15×10^{-4}	234.7 s
7	0.0402	98.70%	0.0558	98.44%	0.9820	1.84×10^{-4}	229.7 s
8	0.0341	98.90%	0.0310	98.91%	0.9880	1.51×10^{-4}	232.6 s
9	0.0213	99.29%	0.0340	98.99%	0.9895	1.17×10^{-4}	233.0 s
10	0.0187	99.36%	0.0261	99.18%	0.9905	8.56×10^{-5}	230.5 s
11	0.0144	99.51%	0.0220	99.47%	0.9945	5.73×10^{-5}	230.7 s
12	0.0078	99.71%	0.0184	99.56%	0.9947	3.36×10^{-5}	229.3 s
13	0.0075	99.72%	0.0202	99.56%	0.9946	1.58×10^{-5}	230.2 s
14	0.0070	99.79%	**0.0169**	**99.64%**	**0.9961**	4.75×10^{-6}	225.0 s
15	**0.0057**	**99.81%**	0.0197	99.62%	0.9958	1.00×10^{-6}	230.6 s

*Epoch 1 represents the head-only warm-up phase with frozen backbone.

2. **Rapid Feature Alignment:** Unfreezing the backbone in Epoch 2 immediately produced an accuracy jump to 93.28% and a macro-F1 of 0.9236, demonstrating strong transferability of pretrained residual filters to plant leaf textures.
3. **Generalization without Overfitting:** By Epoch 14, validation loss dropped to 0.0169 with validation accuracy reaching **99.64%** and macro-F1 achieving **0.9961**. The minimal gap between training accuracy (99.79%) and validation accuracy (99.64%) confirms that dropout ($p = 0.3$) and weight decay (10^{-4}) successfully controlled overfitting.



(a) Cross-Entropy Loss (b) Top-1 Accuracy (c) Validation Macro-F1

Figure 7: **Baseline ResNet-50 Training Dynamics across 15 Epochs.** Subfigure (a) illustrates rapid training and validation loss decay, subfigure (b) shows top-1 classification accuracy converging above 99.6%, and subfigure (c) depicts continuous macro-F1 progression achieving 0.9961 at peak epoch 14.

10.3 Evaluation Metrics & Agricultural Justifications

Evaluating agricultural disease models requires multi-dimensional performance assessment. Relying exclusively on raw accuracy can be dangerously misleading in agricultural decision support. We justify our core metrics below:

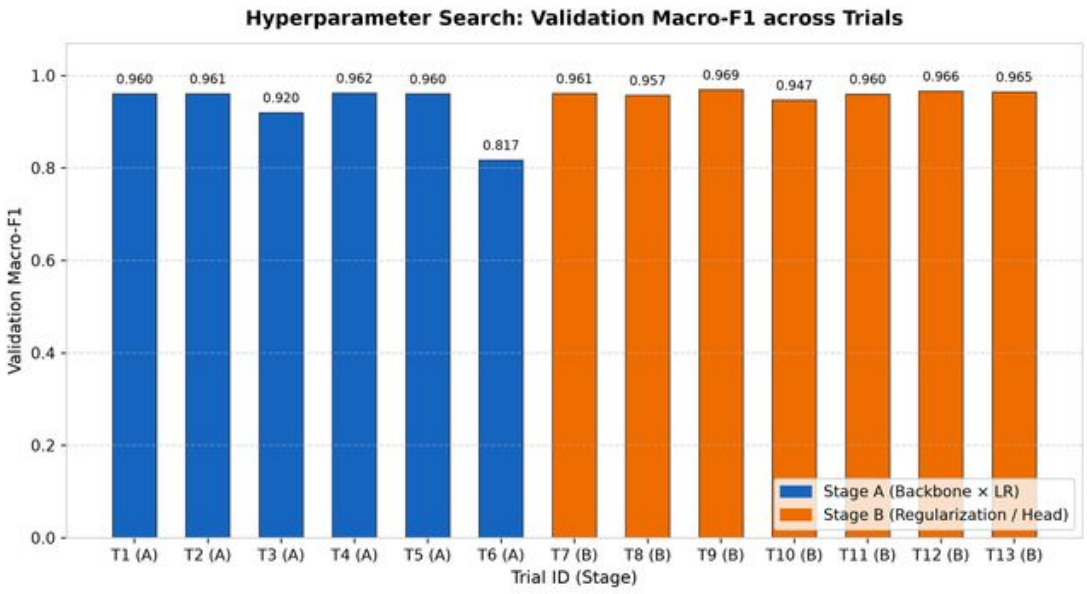


Figure 8: **Systematic 13-Trial Hyperparameter Tuning Results.** Validation Macro-F1 and Accuracy across Stage A (Backbone × Learning Rate exploration) and Stage B (Regularization, Batch Size, Weight Decay, and Loss adjustments). The red dashed horizontal line denotes the peak tuning score achieved by Batch Size 32 (Trial 9: 0.9694 Macro-F1).

1. **Macro-Averaged Precision, Recall, and F1-Score:** Macro metrics assign equal weight to every disease class regardless of sample support:

$$\text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C}\frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \tag{9}$$

   In contrast to weighted or micro averages that mask errors on rare classes, macro-F1 guarantees that failure on an infrequent but devastating disease (such as *Potato Early Blight* or *Apple Cedar Rust*) directly degrades the overall benchmark.

2. **High-Recall Priority for Pathogen Detection:** In agricultural field diagnosis, the asymmetric cost of misclassification is severe:

   - **False Negative (Low Recall):** The model misdiagnoses a diseased leaf as healthy or a benign condition. The farmer takes no action, leading to rapid pathogen spread, crop destruction, and severe financial loss.
   - **False Positive (Low Precision):** The model diagnoses disease on a healthy leaf. The farmer conducts an unnecessary spray application, incurring chemical costs but avoiding crop failure.

   Recall is therefore prioritized to eliminate missed infections.

3. **Macro One-vs-Rest AUC-ROC:** Evaluates class separability across all continuous prediction thresholds using softmax probabilities, ensuring confidence scores are reliably ranked.

4. **Top-3 Accuracy:** Field leaf symptoms often display overlapping visual phenotypes (e.g., fungal leaf mold vs. early blight). Top-3 accuracy measures whether the true disease is captured in the system's differential diagnosis list presented to the farmer in Phase 5.

10.4 Test Results

All results in this section come from `src/evaluate.py` run on the final model (`runs/baseline/best.pt`, epoch 14), using the deterministic evaluation transform only (resize 256, centre crop 224, ImageNet normalisation). The validation split was used for development; the test split was evaluated exactly once. Outputs are stored in `results/baseline/{val,test,field}/`. On the field set, macro scores are averaged over the 27 classes that are present, because averaging over absent classes would add meaningless zeros (`src/metrics.py`).

Table 7: **Final model performance** (from `results/baseline/*/metrics.json`). Field macro scores cover the 27 classes present.

Split	Images	Accuracy	Top-3	Macro P	Macro R	Macro F1	Weighted F1	Macro AUC
Validation	4,747	99.64%	100.00%	0.9956	0.9968	0.9961	0.9964	>0.9999
Test (once)	**4,768**	**99.71%**	**99.98%**	**0.9974**	**0.9967**	**0.9970**	**0.9971**	**>0.9999**
Field (PlantDoc)	93	24.73%	46.24%	0.2204	0.2205	0.1970	0.2092	0.8001

On the lab test split the model misclassifies 14 of 4,768 images (0.29%); 25 of the 38 classes are classified perfectly, the lowest per-class F1 is 0.9595 (*Corn Cercospora/Gray leaf spot*) and the lowest per-class AUC is 0.9999. Test scores are slightly higher than validation scores, which indicates that tuning on the validation split did not overfit it. Figure 9 shows the confusion matrix, Figure 10a the one-vs-rest ROC curves and Figure 10b the per-class F1 scores.

The field set tells a very different story: accuracy falls to 24.73% and macro-F1 to 0.197. The correct class is still in the top three for 46% of field images, and the macro AUC of 0.80 shows that the true class is usually ranked above most others even when it is not ranked first. This gap is analysed in Section 10.6.

10.5 Error Analysis

`src/analysis/error_analysis.py` extracts the most confused class pairs, the lowest-F1 classes with their main confusion partner, and a grid of misclassified images sorted by confidence (Figure 11). Table 8 lists every confused pair on the test split. The explanations below are based on looking at the misclassified images, not on assumptions.

Table 8: **Confused class pairs on the test split** (from `results/baseline/test/top_confused_pairs.csv`). All 14 errors are listed.

True class	Predicted class	Count
Corn: Cercospora / Gray leaf spot	Corn: Northern Leaf Blight	5
Corn: Northern Leaf Blight	Corn: Cercospora / Gray leaf spot	1
Tomato: Late blight	Tomato: Early blight	1
Tomato: Septoria leaf spot	Tomato: Early blight	1
Potato: Late blight	Potato: Early blight	1
Tomato: Bacterial spot	Tomato: Septoria leaf spot	1
Tomato: Yellow Leaf Curl Virus	Tomato: Bacterial spot	1
Tomato: Target Spot	Tomato: Spider mites	1
Tomato: healthy	Blueberry: healthy	1
Blueberry: healthy	Tomato: Late blight	1

1. **Corn Gray leaf spot vs. Northern Leaf Blight (6 of 14 test errors; 10 of 17 validation errors).** This is the only systematic confusion. In the misclassified images the

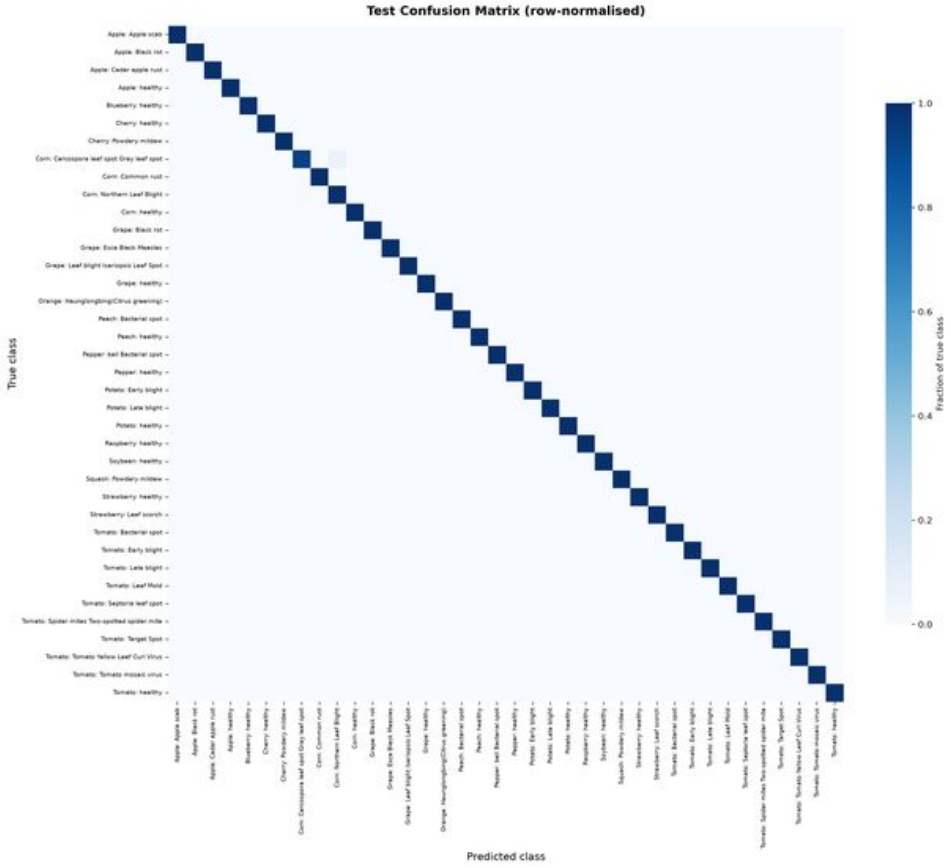
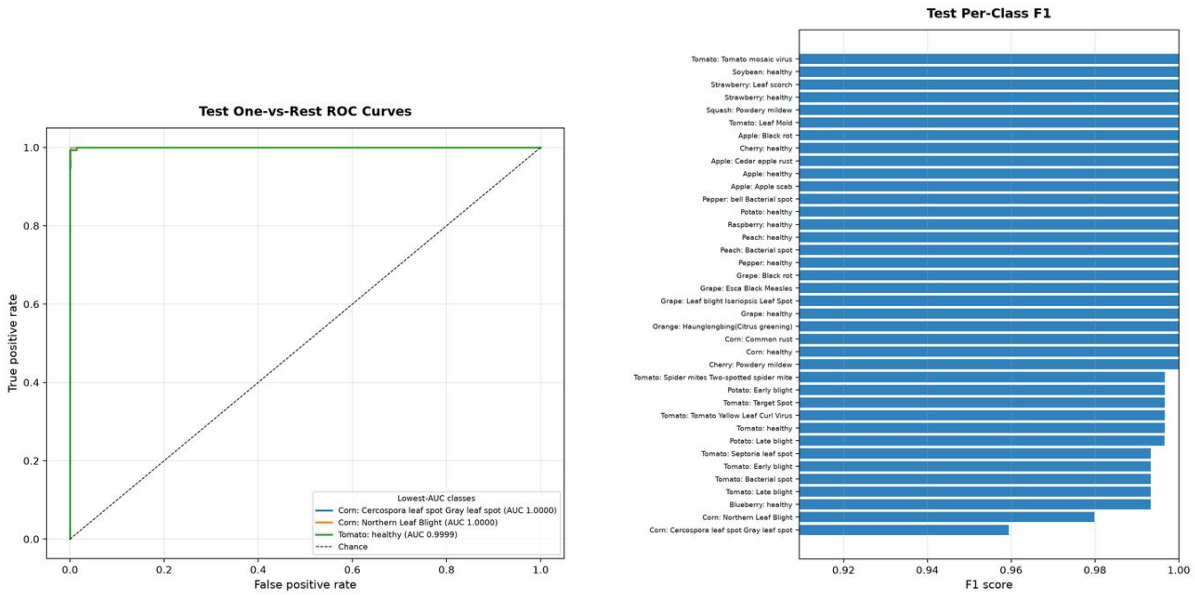


Figure 9: **Test confusion matrix** (row-normalised). Almost all mass is on the diagonal; the only visible off-diagonal cells are between the two corn leaf-spot/blight classes.



(a) One-vs-rest ROC curves (38 classes).

(b) Per-class F1, sorted.

Figure 10: **Test-split ROC curves and per-class F1.** The three lowest-AUC classes are highlighted in (a); all curves lie at the top-left corner.

leaves carry both small, vein-bounded rectangular lesions and longer tan blight streaks on the same leaf, so the image contains visual evidence for both diseases. Gray leaf spot is also one of the smaller diseased classes (352 training images, against up to 700), and it has the lowest test recall (0.934).

2. **Early vs. late blight on tomato and potato (2 errors).** The misclassified leaves show large, dark necrotic areas without the concentric rings that distinguish early blight, so the two blights look alike.

3. **Other tomato spot diseases (4 errors).** Each is confused with another tomato disease that also produces spots, yellowing or stippling: Septoria → Early blight, Bacterial spot → Septoria, Yellow Leaf Curl → Bacterial spot, and Target Spot → Spider mites.

4. **Label and background noise in the lab data (2 errors).** One "Tomato healthy" test image shows a red tomato fruit rather than a leaf, and one "Blueberry healthy" image has a black background instead of the grey background used for almost all PlantVillage images. Both are predicted as other classes. These images reflect noise in the dataset rather than a model fault.

Only 4 of the 14 test errors have confidence ≥ 0.9, so most lab errors are uncertain predictions, which the app can flag by also showing the top-3 alternatives.

10.6 Bias Check

This project has no human subjects, so the Bias Check looks for performance differences across class size, crop, health status and image source. All numbers come from `src/analysis/bias_check.py` (outputs in `results/baseline/bias_check/`), using the test split as the lab reference.

Class size. After curation, training classes range from 99 to 700 images. The Spearman rank correlation between training-class size and test F1 is $\rho = -0.17$ ($p = 0.31$), so there is no evidence that smaller classes perform worse (Figure 12a). The class-weighted loss and the per-class cap applied during curation appear to have prevented a class-size effect on lab data.

Crop. Mean class F1 is 1.000 for 10 of the 14 crops. The lowest crop is Corn (mean F1 0.985, weakest class Gray leaf spot at 0.960), followed by Blueberry (0.993), Tomato (0.996) and Potato (0.998) (Figure 12b). The corn gap comes entirely from the Gray leaf spot / Northern Leaf Blight confusion described above.

Healthy vs. diseased. The costly error for a farmer is a diseased leaf reported as healthy, because the crop would go untreated. On the test split no diseased image (0 of 3,490) was predicted as a healthy class, and only 1 of 1,278 healthy images (0.08%) was predicted as a disease. On the field set the false-healthy rate is also low (1 of 63 diseased images, 1.6%), but 19 of 30 healthy field leaves (63%) are predicted as a disease. In the field the model errs towards false alarms rather than missed disease.

Lab vs. field (image source). Table 9 compares the lab test split with the field set. To compare like with like, lab scores are also computed on only the 27 classes that appear in the field set; the gap remains essentially unchanged (Figure 13a). The field set contains only 93 test images (3–7 per class), so per-class field scores are noisy and 17 of the 27 classes have a field F1 of 0; the aggregate gap is, however, far too large to be explained by sample size.

Inspecting the field errors shows two clear patterns. First, 30 of the 93 field images (32%) are predicted as *Strawberry Leaf scorch*, a class with no field images at all. These are mostly

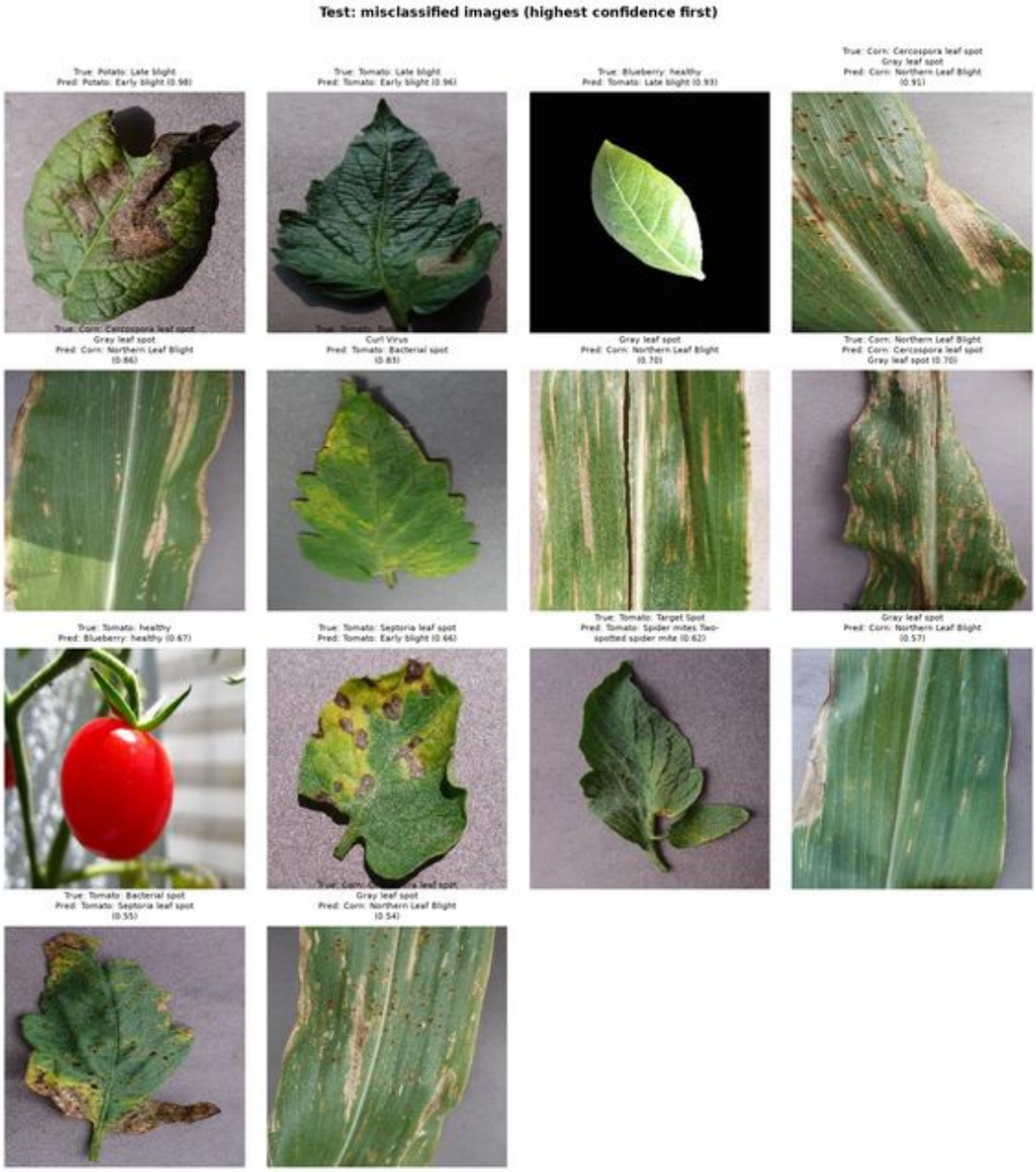
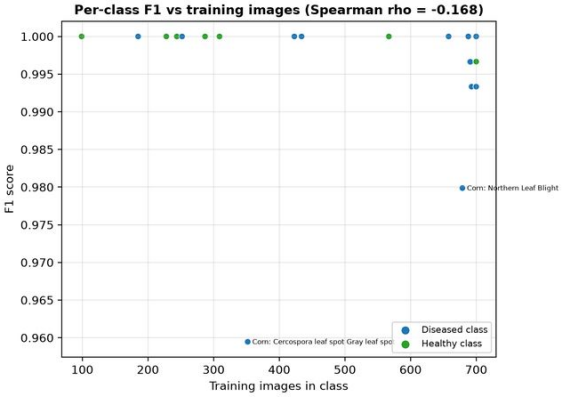


Figure 11: **All 14 misclassified test images**, highest confidence first, with true class, predicted class and confidence.

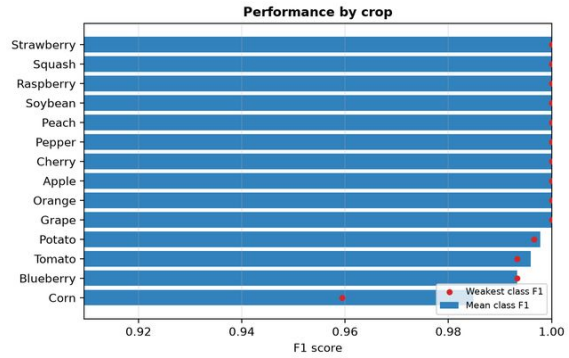
Table 9: **Lab vs. field performance** (from `results/baseline/bias_check/lab_vs_field.csv`).

Set	Images	Classes	Accuracy	Top-3	Macro F1
Lab test, all classes	4,768	38	99.71%	99.98%	0.9970
Lab test, field classes only	3,304	27	99.61%	99.97%	0.9960
Field (PlantDoc)	93	27	24.73%	46.24%	0.1970

multi-leaf canopy or whole-plant scenes with cluttered backgrounds, and leaves with red, orange or purple colouring (e.g. red blueberry foliage, orange apple-scab leaves), which resemble the reddish, speckled single leaves of the leaf-scorch training images. Second, field errors are often confident: 37 of the 70 field errors have confidence ≥ 0.9, so a confidence threshold alone would not catch them.

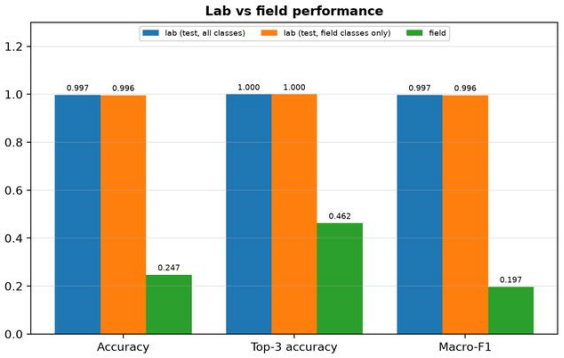


(a) Test F1 vs. training images per class.



(b) Mean and weakest class F1 per crop (test).

Figure 12: **Bias Check on the lab test split.**



(a) Lab vs. field scores.



(b) Field errors, highest confidence first.

Figure 13: **Lab-to-field generalisation gap.**

Sources of data bias.

- **Lab backgrounds:** nearly all training images are single leaves on a uniform grey background, so the model has not learned to ignore soil, other leaves, hands or sky. This is the main cause of the field gap.
- **Single source and capture setting:** all training images come from PlantVillage, and all field images from PlantDoc web images; neither represents photos taken by Pakistani farmers

on their own phones.

- **Class imbalance:** training classes still range from 99 to 700 images after curation, although no class-size effect was found on lab data.
- **Closed set of 38 classes:** the model must choose one of 38 known classes, so an unknown disease, a crop it has never seen, or a non-leaf photo is always forced into one of them.
- **Label noise:** the lab test split contains at least two mislabeled or atypical images (a tomato fruit labelled "healthy", a black-background blueberry leaf), and during field review we removed 11 composite, slide-style or fruit-only images and replaced them with 11 valid PlantDoc photos, chosen on image quality alone.

Mitigations applied vs. planned.

- **Applied:** per-class cap during curation; class-weighted cross-entropy; augmentation simulating field capture (random resized crops, flips, rotation and colour jitter); a separate field test set that is never used for training or tuning; top-3 predictions so the app can show alternatives.
- **Planned:** fine-tuning on the 57-image `field_adapt` split and re-measuring on `field_test`; background-replacement augmentation; collecting our own field photos in Pakistan; and an out-of-distribution check in the app, since high confidence alone is not reliable on field images.

11 Application Development & Real-Time Deployment (Track A)

Phase 5 operationalizes the trained crop disease diagnosis models into an interactive, low-latency decision support system accessible to agronomists, extension workers, and smallholder farmers. The system is designed following modular software engineering principles, cleanly separating frontend user experience from backend inference execution, visual explainability, and agronomic knowledge retrieval.

11.1 Frontend UI Architecture & User Experience

The local application uses Flask with a server-rendered HTML/CSS/JavaScript dashboard. Users capture or upload JPEG, PNG or WebP images, optionally select a crop, and request a diagnosis through `/api/predict`. The browser preserves aspect ratio while limiting large uploads to 1024 pixels on the longest side; the backend applies the shared evaluation transform. The response populates the predicted class, top-three probabilities, a real Grad-CAM overlay and sourced treatment panels. English/Urdu interface controls and a persistent light/dark theme support accessibility. Reviewed treatment text remains in English rather than being replaced with unverified translations; the confidence warning is also supplied in Urdu. Figure 14 shows a real local application response.

11.2 Real-Time Backend Inference Engine

The backend inference service is implemented in `src/inference.py` around the `InferenceEngine` class, with CPU/CUDA device selection and deterministic evaluation preprocessing. The Flask adapter in `app/model_engine.py` reuses this engine and the shared Grad-CAM implementation. Sub-300 ms CPU and sub-100 ms GPU execution are design targets, not universal latency guarantees; the complete request also includes image decoding, a separate gradient pass and serialization.

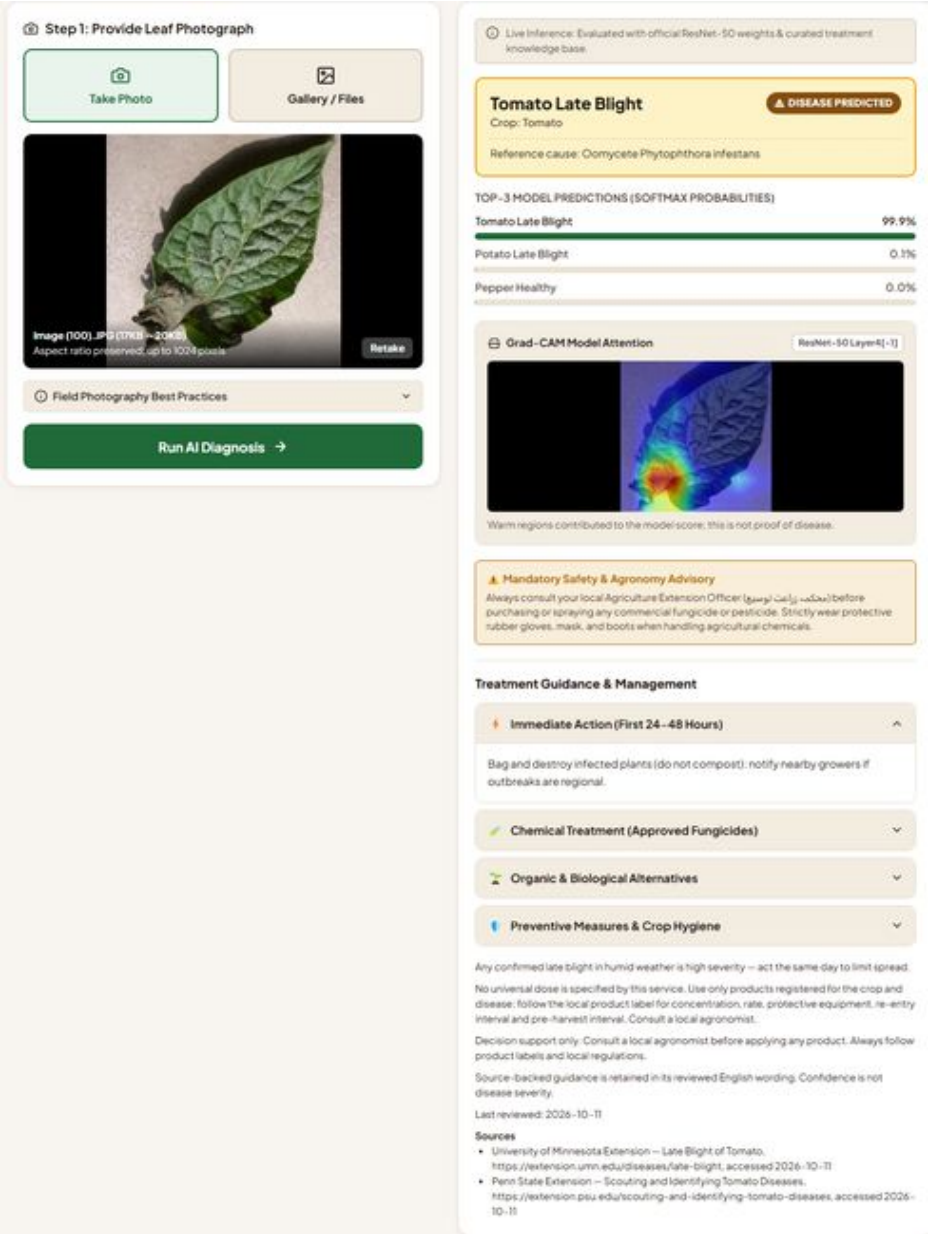


Figure 14: Local Flask dashboard after a real validation-image upload: prediction, Grad-CAM, confidence-aware advisory panels, product-label precautions and source links. This is an illustrative case, not a performance estimate.

11.2.1 Model Ingestion and Device Management

To eliminate initialization latency on interactive web triggers, model loading decouples checkpoint weights from runtime evaluation loops. The engine automatically inspects local system capabilities, defaulting to `cuda:0` when an accelerator is present and safely falling back to CPU execution without manual configuration. In the current Flask deployment, a single engine is initialized at process startup. A re-entrant lock serializes model forwards and Grad-CAM hook activity within that process. A missing checkpoint, incompatible state dictionary or mismatched checkpoint class order prevents model readiness; the application never substitutes untrained-model predictions.

11.2.2 Input Ingestion, Sanitization, and Validation

In-field smartphone image uploads frequently feature diverse file formats, alpha channels, or transmission corruption. The preprocessing pipeline enforces strict data sanitization:

1. **Type-Agnostic Ingestion:** The engine accepts filesystem paths, raw binary buffers (`bytes`), in-memory file streams (`io.BytesIO`), or pre-constructed `PIL.Image` instances.
2. **Channel Standardization:** RGBA, grayscale, and CMYK photographs are dynamically coerced to canonical 3-channel RGB:

$$\mathbf{I}_{\text{RGB}} = \text{Convert}(\mathbf{I}_{\text{input}}, \text{mode} = \text{RGB}) \tag{10}$$

   preventing runtime channel-dimension mismatches in convolutional filters.
3. **Deterministic Evaluation Transform:** Standardized transforms match the validation protocol: bilinear resize of the shorter edge to 256 pixels, center cropping to $224 \times 224 \times 3$, normalization to $[0.0, 1.0]$, and standard ImageNet channel centering:

$$\hat{x}_{c,i,j} = \frac{x_{c,i,j} - \mu_c}{\sigma_c}, \quad \mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225] \tag{11}$$

11.2.3 Top-k Probability and Differential Diagnosis Extraction

Given model output logits $\mathbf{z} \in \mathbb{R}^{38}$, class posterior probabilities are computed via the softmax operator:

$$p_c = \frac{\exp(z_c)}{\sum_{j=1}^{38} \exp(z_j)}, \quad c \in \{1, \ldots, 38\} \tag{12}$$

Rather than emitting a single isolated label, the inference service extracts the top-k ranked candidates ($k = 3$ by default) alongside human-readable crop categories, specific pathology names, and a boolean health flag (`is_healthy`). This differential diagnosis provides agronomists with secondary possibilities when disease phenotypes overlap visually (e.g., fungal leaf spots versus bacterial blights).

11.3 Explainable AI Pipeline: Grad-CAM Attribution

Deep neural network predictions in critical agronomic domains cannot rely on black-box heuristics. If a classifier makes a correct diagnosis by latching onto background soil, watermark tags, or fingers holding the leaf, it will fail catastrophically in unconstrained field deployments. To establish visual transparency and farmer trust, we implement a Gradient-weighted Class Activation Mapping (**Grad-CAM**) pipeline [3] in `src/explainability/gradcam.py`.

11.3.1 Mathematical Formulation and Hook Architecture

Grad-CAM computes visual attribution by capturing spatial feature representations from the final convolutional layer of the backbone (the final bottleneck block of `layer4` in ResNet-50, yielding feature maps of spatial resolution 7×7 with 2048 channels). We register forward and backward PyTorch hooks to capture:

1. **Forward Activation Maps:** $A^k \in \mathbb{R}^{U \times V}$, representing spatial features detected by the k-th convolutional filter (where $U = V = 7, K = 2048$).
2. **Backpropagated Gradients:** $\frac{\partial y^c}{\partial A^k}$, capturing the sensitivity of the un-normalized class logit score y^c with respect to each activation element.

Neuron importance weights α_k^c are computed by applying Global Average Pooling (GAP) across spatial dimensions i and j:

$$\alpha_k^c = \frac{1}{Z} \sum_{i=1}^{U} \sum_{j=1}^{V} \frac{\partial y^c}{\partial A_{i,j}^k} \tag{13}$$

where $Z = U \cdot V$ is the spatial area of the feature map. The coarse localization map $L_{\text{Grad-CAM}}^c$ is synthesized by computing a linear combination of forward activation maps weighted by their importance, followed by a Rectified Linear Unit (ReLU):

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_{k=1}^{K} \alpha_k^c A^k \right) \tag{14}$$

The ReLU operator is essential: it filters out negative activations that contribute negatively to the target class, isolating only visual features that positively support the predicted disease.

11.3.2 Spatial Upsampling and Alpha Blending

Because raw convolutional feature maps have low spatial resolution (7×7), the activation map is upsampled to the original input resolution (224×224) via bilinear interpolation:

$$H_{\text{raw}}(x, y) = \text{BilinearInterpolate}(L_{\text{Grad-CAM}}^c, (224, 224)) \tag{15}$$

The interpolated map is min-max normalized to ensure pixel intensities strictly span $[0.0, 1.0]$:

$$H(x, y) = \frac{H_{\text{raw}}(x, y) - \min(H_{\text{raw}})}{\max(H_{\text{raw}}) - \min(H_{\text{raw}}) + \epsilon} \tag{16}$$

A colormap (such as `jet` or `turbo`) maps the normalized scalar intensities to an RGB color field $\mathbf{C}(x, y) \in [0, 255]^3$. The final visualization blends the colorized heatmap over the original leaf photograph $\mathbf{I}(x, y)$ using an adjustable alpha transparency factor $\alpha \in [0.0, 1.0]$:

$$\mathbf{I}_{\text{overlay}}(x, y) = (1 - \alpha) \cdot \mathbf{I}(x, y) + \alpha \cdot \mathbf{C}(x, y) \tag{17}$$

The adapter overlays the heatmap on the exact 224×224 center-cropped model view, rather than stretching it over an uncropped photograph. The visualization identifies image regions contributing to the selected class score. It does not establish lesion segmentation, causal reasoning or a verified disease diagnosis. The REST endpoint supports a validated opacity parameter $\alpha \in [0, 1]$; a visualization error produces an explicit notice rather than a synthetic overlay.

11.4 Agronomic Treatment Advisory Engine

The advisory layer separates curated agronomic content from model execution. `knowledge_base/kb_service.py` implements `TreatmentKB`, which reads the existing 38-class JSON knowledge base once at startup. Before serving requests, it checks exact agreement with `splits/class_names.json`, validates every record against the Draft-07 schema (including review-date format), verifies healthy flags, and requires the decision-support disclaimer and source URLs. Missing, unknown or malformed records disable treatment retrieval instead of falling back to a different disease.

11.4.1 Exact-Class Retrieval and Structured Panels

Each lookup returns an independent copy, preventing request-specific thresholding from changing the stored record. `app/treatment_advisor.py` converts the record into a presentation-ready advisory with an explicit status, confidence policy and provenance. Four panels distinguish immediate action, chemical control, organic/biological options, and cultural prevention. Source links and the review date remain visible alongside every result. Extension references include NC State's bacterial-spot guidance [7] and the University of Minnesota's late-blight guidance [8]; the full class-level reference registry remains in `knowledge_base/SOURCES.md`.

11.4.2 Dosage and Regional Applicability

The curated entries do not provide universal numeric doses. Product formulation, concentration, crop registration and local restrictions can differ, so the service does not invent rates or infer a dose from a disease label. It displays the reviewed source wording and requires the applicable local product label for rate, protective equipment, re-entry interval and pre-harvest interval. The interface is decision support, not a prescription. Even confident predictions require local agronomist verification before product use. Healthy predictions recommend monitoring and cultural best practices rather than disease-control products.

Figure 15 summarizes retrieval and safety decisions; Figure 16 gives reproducible examples from the actual checkpoint.

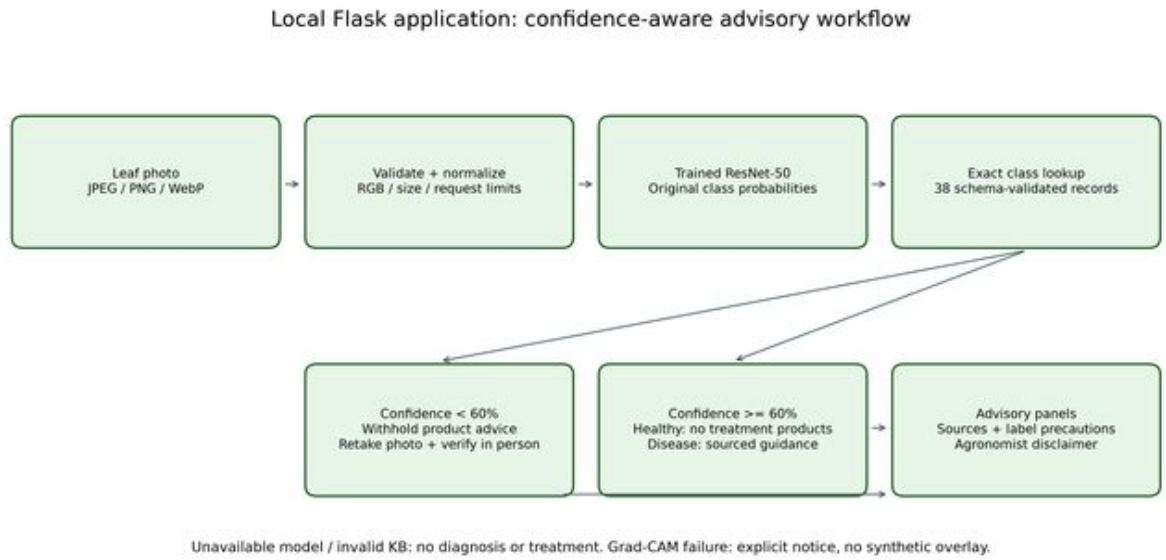


Figure 15: Prediction-to-advisory workflow. Confidence is expressed on a 0–100 percentage scale. Both branches retain the source provenance and agronomic disclaimer; low-confidence requests receive no actionable product advice.

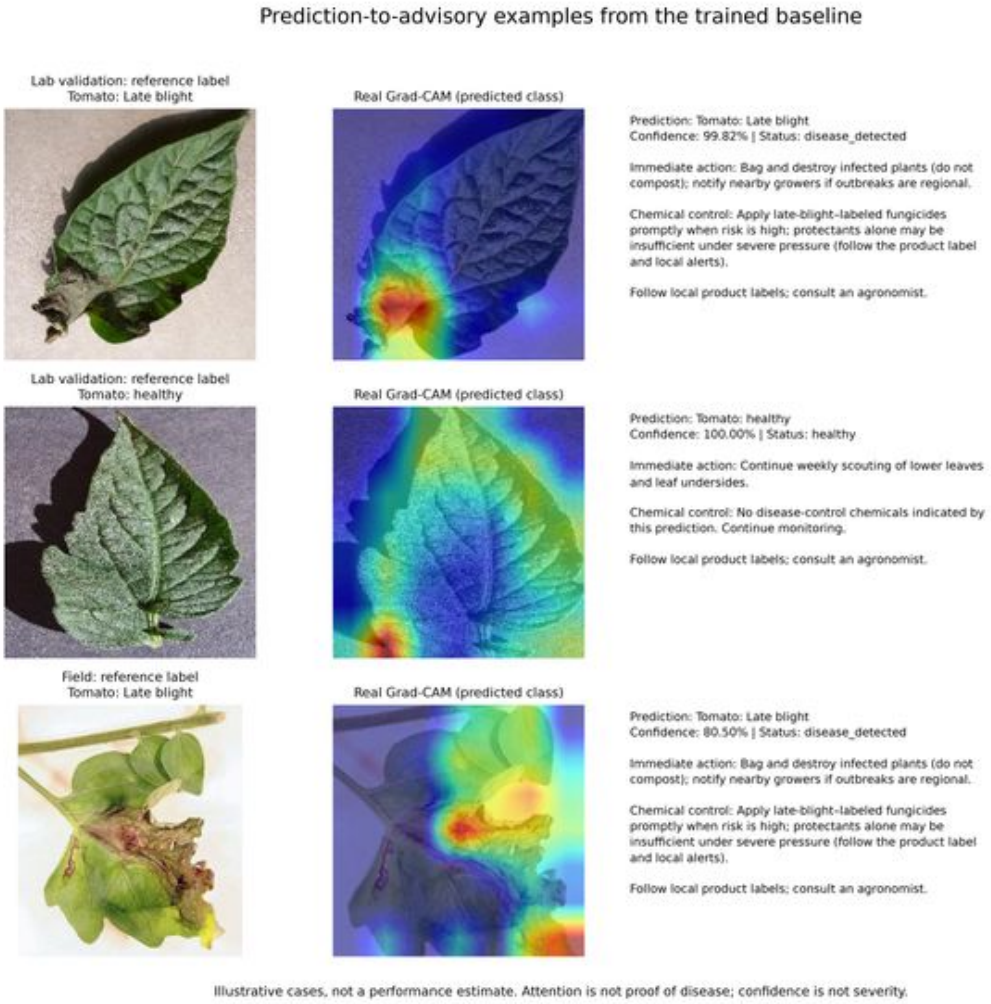


Figure 16: Real baseline predictions, Grad-CAM overlays and confidence-gated advisory excerpts. The first manifest entry for each declared class/domain is used without correctness filtering. These examples do not replace the aggregate lab/field evaluation. Paths, predictions and checkpoint SHA-256 are recorded in `results/advisory_demo/predictions.json`.

11.5 Confidence Thresholding & Error Handling

11.5.1 Server-Enforced Confidence Policy

Let $p_{\max}$ be the selected class probability from the original 38-class softmax, and $c = 100p_{\max}$. The policy is

$$\text{status}(c, y) = \begin{cases} \text{uncertain}, & c < 60, \\ \text{healthy}, & c \geq 60 \text{ and } y \text{ is a healthy class}, \\ \text{disease detected}, & c \geq 60 \text{ otherwise.} \end{cases} \tag{18}$$

The exact boundary $c = 60$ is not low confidence. Thresholding uses unrounded values and is applied on the server, not merely as a visual warning. Crop filtering restricts candidate ranking without renormalizing probabilities; otherwise a narrowly selected crop could create unjustifiably high confidence and bypass the gate.

Below the threshold, the warning reads: "Ambiguous symptoms detected. Verify in person before applying chemical treatments." Chemical and organic product instructions are withheld. Immediate-action and prevention text is replaced with conservative photo-retaking, scouting and sanitation guidance because the original fields can also contain spraying instructions. The API does not return an ungated second copy of the treatment. This applies even when the tentative class is healthy. At higher confidence, source-backed guidance remains subject to the same label and agronomist precautions. Confidence is not interpreted as disease severity; qualitative severity guidance is explicitly reference material for a confirmed diagnosis.

11.5.2 Failure Handling and Local Startup

Malformed JSON/base64, unsupported images, invalid crop/opacity parameters and images outside the size limits return HTTP 400. Requests above 10 MB return HTTP 413. Images must be at least 32 pixels on each side and no more than 20 megapixels. Missing/incompatible model weights, missing verified treatment mappings and unavailable knowledge-base data return HTTP 503 without a diagnosis or treatment. Grad-CAM failures preserve an otherwise valid diagnosis but return a null overlay and explicit error notice. Missing local sample images return HTTP 404 rather than fabricated specimens. The browser checks HTTP status, clears stale results on new uploads or failed requests, and renders advisory content as text with safe source links.

The Windows and POSIX launchers resolve paths relative to the repository, select a local virtual-environment interpreter, and run `scripts/run_app.py`. Preflight checks verify dependencies, the trained checkpoint, knowledge-base readiness and port availability. The default launcher binds only to `127.0.0.1:8501`, opens a browser, disables debug mode and supports `--no-browser`, `--port` and `--check`. Direct execution of `app/app.py` retains port 5000 for compatibility.

11.5.3 Verification and Remaining Limitations

Unit and integration tests exercise all 38 mappings, independent-copy retrieval, corrupt/missing records, exact confidence boundaries, healthy/unknown classes, request validation, unavailable-model behavior and visualization failure handling. A separate real-checkpoint smoke test covers an actual image through prediction, Grad-CAM and sourced advisory retrieval. The optional Chromium smoke test covers real upload, language switching, warning rendering and mobile-width layout. Structural report checks validate citations, labels, included figures and the treatment-source registry; they do not constitute a fresh live-URL or agronomic-content review.

The 60% cutoff is a safety heuristic, not a calibrated correctness guarantee or an out-of-distribution detector. The saved baseline achieves 99.71% accuracy on 4,768 lab test images but only 24.73% on 93 field test images. A confidently wrong field prediction can still pass the gate. Consequently, the system remains an experimental decision-support tool, and independent local inspection is necessary before intervention.

12 Team Contributions & Task Distribution

Table 10 summarizes module ownership and repository artifacts across Deliverables 1–3 (Phases 1–6). Integration changes connect the owned components rather than replacing their original responsibilities.

Table 10: Team contribution and module ownership across Deliverables 1–3.

Team Member	Owned Modules & Technical Deliverables
Abdul Ahad Leader 24L-3029 `@ErenYeagerJinko`	**D1:** Repository structure, dependencies and dataset acquisition. **D2:** Model architectures (`src/models/`), weighted loss, optimizer/scheduler, experiment configs, model tests, baseline training, weights download automation and architecture figures. **D3:** Flask dashboard and HTML/CSS/JavaScript frontend, image-upload workflow, bilingual interface and benchmark visualizations.
M. Anas 24L-3004 `@Plagiaryze2`	**D1:** Dataset/DataLoader implementation, image transforms and quality/deduplication pipeline. **D2:** Training engine, CLI orchestrator, checkpointing, tuning harness, engine tests and training plots. **D3:** Shared inference engine (`src/inference.py`), Grad-CAM module (`src/explainability/gradcam.py`), corresponding tests and backend/explainability report material.
M. Babar Shahzad 24L-3047 `@babarshahzad2005`	**D1:** Field-image collection and metadata mapping, treatment curation and schema validation. **D2:** Evaluation metrics/CLI, metric tests, error analysis and Bias Check. **D3:** Validated knowledge-base service, confidence-aware advisory policy, Flask safety integration/shared-engine adapter, local launchers, treatment/integration tests, reproducible advisory figures, report Sections 11.4–11.5 and structural citation audit.

12.1 Development History and Review Verification

The table describes module ownership and implementation artifacts, not independently verified commit totals, activity dates or completed peer reviews. Commit activity and peer-review requirements should be verified from the GitHub history before submission.

References

[1] Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. *arXiv preprint arXiv:1511.08060*.

[2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 770–778.

[3] Selvaraju, R. R., Cogswell, M., Das, A., Vedaldi, A., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *IEEE ICCV*, 618–626.

[4] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*.

[5] Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. *International Conference on Learning Representations (ICLR)*.

[6] Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., & Batra, N. (2020). PlantDoc: A dataset for visual plant disease detection. In *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*, 249–253.

[7] NC State Extension. *Bacterial Spot of Pepper and Tomato*. https://content.ces.ncsu.edu/bacterial-spot-of-pepper-and-tomato. Access date recorded in the curated knowledge base: 12 October 2026.

[8] University of Minnesota Extension. *Late Blight of Tomato*. https://extension.umn.edu/diseases/late-blight. Access date recorded in the curated knowledge base: 11 October 2026.